

Technical Documentation: Llama 3.3 Powered Flask AI Chatbot

Shayan Ali

July 14, 2026

Contents

1	Introduction	3
2	System Architecture	3
2.1	Backend Framework	3
2.2	Intelligence Layer (Groq API)	3
2.3	Frontend Interface	3
3	Project Directory Structure	3
4	Implementation Details	3
5	Installation and Local Execution	6
5.1	Prerequisites	6
5.2	Environment Initialization	6
5.3	Running the Application	6
6	Troubleshooting Guide	6
6.1	ModuleNotFoundError: No module named 'flask'	6
6.2	Port Already in Use (Address already in use)	6

1 Introduction

This document outlines the technical architecture, design, installation, and deployment procedures for the Flask-based artificial intelligence chatbot. The system is engineered as a lightweight, low-latency web application leveraging the Groq SDK to process conversational queries through state-of-the-art Large Language Models (LLMs), specifically the Llama 3.3 70B variant.

2 System Architecture

The application follows a standard client-server architecture model, divided into distinct backend and frontend components.

2.1 Backend Framework

The core server is constructed using Flask, a micro web framework written in Python. Flask is utilized to map client-side HTTP requests to backend controller functions and handle secure, asynchronous routing to external APIs.

2.2 Intelligence Layer (Groq API)

Instead of hosting a heavy open-source model locally, processing is delegated to the Groq Cloud API. Groq utilizes Language Processing Units (LPUs) to deliver near-instant responses. The backend uses the official Python `groq` library to construct completion payloads and manage security handshakes.

2.3 Frontend Interface

The client interface consists of a responsive single-page application built using HTML5, CSS3, and native JavaScript. It communicates with the backend via AJAX `fetch` POST operations, allowing real-time screen updates without requiring full page refreshes.

3 Project Directory Structure

The repository organizes configuration, backend logical handlers, and frontend markup templates cleanly:

```
chatbot/
  app.py           % Flask application initialization and route handling
  templates/
    index.html     % User interface HTML layout and UI scripting
  venv/            % Isolated local Python virtual environment dependencies
```

4 Implementation Details

Below is the clean, secured backend implementation of the `app.py` script. It contains the core API connectivity logic, session initialization, and robust system-level error handling.

```

1 import os
2 from flask import Flask, render_template, request, jsonify
3 from groq import Groq
4
5 app = Flask(__name__)
6
7 # Initialize the Groq client securely
8 # System environment variables are checked first, with a hardcoded
   local fallback key
9 API_KEY = os.environ.get("GROQ_API_KEY", "
   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")
10 client = Groq(api_key=API_KEY)
11
12 MODEL = "llama-3.3-70b-versatile"
13
14 @app.route('/')
15 def home():
16     """Renders the main chatbot interface page."""
17     return render_template('index.html')
18
19 @app.route('/chat', methods=['POST'])
20 def chat():
21     """Handles sending the payload to Groq and returning the text."""
22     try:
23         if not request.is_json:
24             return jsonify({'error': 'Request must be JSON'}), 400
25
26         data = request.json
27         messages = data.get('messages', [])
28
29         if not messages:
30             return jsonify({'error': 'No messages provided'}), 400
31
32         # Inject system prompt at the beginning if not present
33         if messages[0].get('role') != 'system':
34             messages.insert(0, {
35                 "role": "system",
36                 "content": "You are a highly capable AI assistant. Give
   concise, well-structured answers."
37             })
38
39         # Fetch the completion from Groq
40         completion = client.chat.completions.create(
41             messages=messages,
42             model=MODEL,
43             stream=False
44         )
45
46         reply = completion.choices[0].message.content
47         return jsonify({'reply': reply})
48
49     except Exception as e:
50         import traceback
51         traceback.print_exc()
52         return jsonify({'error': str(e)}), 500
53
54 if __name__ == '__main__':

```

```
55 app.run(debug=True, port=5000)
```

Listing 1: app.py Source Code

5 Installation and Local Execution

5.1 Prerequisites

Ensure that Python 3.12 or newer and Node.js are installed on your Linux environment.

5.2 Environment Initialization

Execute the following commands in sequence to clone the project, isolate the workspace dependencies, and install the necessary libraries:

```
1 # Clone repository
2 git clone https://github.com/YOUR_USERNAME/chatbot.git
3 cd chatbot
4
5 # Establish isolated virtual environment
6 python3 -m venv venv
7
8 # Activate virtual environment
9 source venv/bin/activate
10
11 # Install third-party requirements
12 pip install flask groq httpx
```

5.3 Running the Application

To launch the server locally, confirm your virtual environment is active (indicated by (venv) prepending your terminal line) and start the listener:

```
1 python3 app.py
```

Once initiated, open a web browser and navigate to `http://127.0.0.1:5000`.

6 Troubleshooting Guide

6.1 ModuleNotFoundError: No module named 'flask'

This error occurs when the Python executable cannot find Flask in its current environment path.

- **Cause:** The terminal session has been closed or refreshed, causing the local virtual environment to deactivate.
- **Solution:** Re-run `source venv/bin/activate` in your active terminal before booting the server.

6.2 Port Already in Use (Address already in use)

This occurs when a ghost process fails to release port 5000 during a server restart.

- **Cause:** The previous Flask thread was not terminated properly.
- **Solution:** Free the locked port manually via your terminal using:

```
1 fuser -k 5000/tcp
2
```

Alternatively, modify the driver port within the entry point of `app.py` (e.g., change port parameter to 5001).